

PYTHON PROGRAMMING LAB - UNIT 5

24. Program to Check Whether a JSON String Contains a Complex Object

Aim

To write a Python program to check whether a JSON string contains a complex object such as a list or a nested dictionary.

Algorithm

1. Start the program.
2. Import the `json` module.
3. Define a JSON string.
4. Parse the JSON string into a Python object using `json.loads()`.
5. Check if any value in the parsed object is of type `dict` or `list`.
6. If found, display that the JSON contains a complex object; otherwise, display that it does not.
7. Stop the program.

Source Code

```
# Program to check if a JSON string contains complex object

import json

# Example JSON string
json_str = '{"name": "Ravi", "age": 21, "subjects": ["Math",
"Physics"], "address": {"city": "Hyderabad"}}'

# Parse JSON string
data = json.loads(json_str)
```

```
# Function to check for complex objects
def contains_complex(obj):
    if isinstance(obj, dict):
        for value in obj.values():
            if isinstance(value, (dict, list)):
                return True
    elif isinstance(obj, list):
        for item in obj:
            if isinstance(item, (dict, list)):
                return True
    return False

# Check and display result
if contains_complex(data):
    print("The JSON string contains a complex object.")
else:
    print("The JSON string does not contain any complex object.")
```

Output (Sample Run)

Result

Thus, the Python program to check whether a JSON string contains a complex object was successfully executed and verified.

25. Program to Demonstrate NumPy Arrays Creation Using `array()` Function

Aim

To write a Python program to demonstrate the creation of NumPy arrays using the `array()` function.

Algorithm

1. Start the program.
2. Import the `numpy` module.
3. Create NumPy arrays using the `numpy.array()` function with different types of data:
 - 1D array
 - 2D array (matrix)
 - Array with different data types (int, float)
4. Display the created arrays.
5. Stop the program.

Source Code

```
# Program to demonstrate NumPy array creation using array() function

import numpy as np

# 1D Array
arr1 = np.array([1, 2, 3, 4, 5])
print("1D Array:", arr1)

# 2D Array
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:\n", arr2)
```

```
# Array with float values
arr3 = np.array([1, 2, 3, 4], dtype=float)
print("Array with float values:", arr3)

# Array with strings
arr4 = np.array(["Python", "NumPy", "Programming"])
print("Array with strings:", arr4)
```

Output (Sample Run)

Result

Thus, the Python program to demonstrate NumPy array creation using the `array()` function was successfully executed and verified.

26. Python Program to Demonstrate Use of **ndim**, **shape**, **size**, **dtype**

Aim

To write a Python program that demonstrates the use of NumPy array attributes: **ndim**, **shape**, **size**, and **dtype**.

Algorithm

1. Start the program.
2. Import the **numpy** module.
3. Create NumPy arrays of different dimensions (1D and 2D).
4. For each array, display the following properties:
 - **ndim** → Number of dimensions of the array
 - **shape** → Number of rows and columns
 - **size** → Total number of elements
 - **dtype** → Data type of elements in the array
5. Stop the program.

Source Code

```
# Program to demonstrate use of ndim, shape, size, dtype in NumPy arrays

import numpy as np

# 1D Array
arr1 = np.array([10, 20, 30, 40, 50])
print("1D Array:", arr1)
print("Dimensions (ndim):", arr1.ndim)
print("Shape:", arr1.shape)
```

```
print("Size:", arr1.size)
print("Data type:", arr1.dtype)
print()

# 2D Array
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:\n", arr2)
print("Dimensions (ndim):", arr2.ndim)
print("Shape:", arr2.shape)
print("Size:", arr2.size)
print("Data type:", arr2.dtype)
```

Sample Output

Result

Thus, the Python program demonstrating the use of `ndim`, `shape`, `size`, and `dtype` was successfully executed and verified.

27. Python Program to Demonstrate Basic Slicing, Integer, and Boolean Indexing

Aim

To write a Python program that demonstrates **basic slicing**, **integer indexing**, and **Boolean indexing** in NumPy arrays.

Algorithm

1. Start the program.
2. Import the `numpy` module.
3. Create a 1D NumPy array with some numbers.
4. Perform **basic slicing** to extract a range of elements.
5. Perform **integer indexing** to access specific positions.
6. Perform **Boolean indexing** to filter elements based on conditions.
7. Display results.
8. Stop the program.

Source Code

```
# Program to demonstrate slicing, integer indexing, and Boolean indexing

import numpy as np

# Create a NumPy array
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])

print("Original Array:", arr)

# 1. Basic Slicing
print("\nBasic Slicing (arr[2:6]):", arr[2:6])          # elements from
index 2 to 5
```

```
print("Basic Slicing with step (arr[1:7:2]):", arr[1:7:2]) # elements
from index 1 to 6 with step 2

# 2. Integer Indexing
indices = [0, 2, 5]
print("\nInteger Indexing (arr[[0,2,5]]):", arr[indices]) # pick
specific elements

# 3. Boolean Indexing
bool_mask = arr > 40 # condition
print("\nBoolean Mask (arr > 40):", bool_mask)
print("Boolean Indexing (arr[arr > 40]):", arr[bool_mask])
```

Sample Output

Result

Thus, the Python program successfully demonstrated **basic slicing, integer indexing, and Boolean indexing** in NumPy arrays.

28. Python Program to Find Min, Max, Sum, and Cumulative Sum of Array

Aim

To write a Python program using NumPy to compute the **minimum, maximum, sum, and cumulative sum** of elements in an array.

Algorithm

1. Start the program.
2. Import the `numpy` module.
3. Create a NumPy array with some elements.
4. Use:
 - `np.min()` to find the smallest element.
 - `np.max()` to find the largest element.
 - `np.sum()` to compute the total sum.
 - `np.cumsum()` to compute the cumulative sum.
5. Display results.
6. Stop the program.

Source Code

```
# Program to find min, max, sum, and cumulative sum of array

import numpy as np

# Create an array
arr = np.array([5, 10, 15, 20, 25])

print("Array:", arr)
```

```
# Minimum element
print("Minimum:", np.min(arr))

# Maximum element
print("Maximum:", np.max(arr))

# Sum of elements
print("Sum:", np.sum(arr))

# Cumulative sum
print("Cumulative Sum:", np.cumsum(arr))
```

Sample Output

Result

Thus, the Python program using NumPy to find **min, max, sum, and cumulative sum** of an array was successfully executed and verified.

29. Create a Dictionary with Keys Representing Lists and Convert to Pandas DataFrame

Aim

To write a Python program that creates a dictionary with lists as values, converts it into a Pandas DataFrame, and explores the data using **head()** and **data selection operations**.

Algorithm

1. Start the program.
2. Import the `pandas` library.
3. Create a dictionary with at least five keys where each key contains a list of at least ten values.
4. Convert the dictionary into a Pandas DataFrame using `pd.DataFrame()`.
5. Display the first five rows using the `head()` function.
6. Perform various data selection operations:
 - Select a single column.
 - Select multiple columns.
 - Select specific rows using slicing.
 - Select rows and columns using `.loc[]` and `.iloc[]`.
7. Stop the program.

Source Code

```
# Program to create dictionary, convert to DataFrame and explore data

import pandas as pd

# Step 1: Create dictionary
data = {
```

```
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank",
"Grace", "Hannah", "Ian", "Julia"],
    "Age": [20, 21, 22, 20, 23, 21, 24, 22, 23, 25],
    "Marks": [85, 90, 78, 88, 76, 95, 89, 92, 80, 87],
    "City": ["Delhi", "Mumbai", "Kolkata", "Chennai", "Delhi", "Pune",
"Mumbai", "Delhi", "Chennai", "Kolkata"],
    "College": ["IIT", "NIT", "IIT", "BITS", "DU", "IIT", "NIT", "DU",
"BITS", "NIT"]
}
```

```
# Step 2: Convert dictionary to Pandas DataFrame
df = pd.DataFrame(data)
```

```
# Step 3: Display DataFrame
print("DataFrame:\n", df)
```

```
# Step 4: Apply head() function
print("\nFirst 5 Rows using head():\n", df.head())
```

```
# Step 5: Perform data selection operations
print("\nSingle Column (Name):\n", df["Name"])
print("\nMultiple Columns (Name, Age):\n", df[["Name", "Age"]])
print("\nSelecting first 3 rows:\n", df[:3])
print("\nUsing loc (Rows 0-2, Columns Name & City):\n", df.loc[0:2,
["Name", "City"]])
print("\nUsing iloc (Rows 2-5, first 3 columns):\n", df.iloc[2:6,
0:3])
```

Sample Output

Result

Thus, a dictionary was created, converted into a Pandas DataFrame, and explored using **head()** and various **data selection operations** successfully.

30. Select Any Two Columns from DataFrame and Observe Relationship Using Scatter and Plot

Aim

To select any two columns from a Pandas DataFrame and observe the change in one attribute with respect to another using **scatter** and **line plots** in Matplotlib.

Algorithm

1. Start the program.
2. Import the required libraries: `pandas` and `matplotlib.pyplot`.
3. Create a dictionary and convert it into a DataFrame.
4. Select two columns (e.g., `Age` and `Marks`).
5. Plot a **scatter plot** of Marks vs Age.
6. Plot a **line plot** of Marks vs Age.
7. Display both plots.
8. Stop the program.

Source Code

```
# Program to plot scatter and line chart for DataFrame columns

import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Create dictionary and DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank",
            "Grace", "Hannah", "Ian", "Julia"],
    "Age": [20, 21, 22, 20, 23, 21, 24, 22, 23, 25],
    "Marks": [85, 90, 78, 88, 76, 95, 89, 92, 80, 87],
    "City": ["Delhi", "Mumbai", "Kolkata", "Chennai", "Delhi", "Pune",
            "Mumbai", "Delhi", "Chennai", "Kolkata"]
}
```

```
        "College": ["IIT", "NIT", "IIT", "BITS", "DU", "IIT", "NIT", "DU",
                    "BITS", "NIT"]
    }

df = pd.DataFrame(data)

# Step 2: Select two columns
x = df["Age"]
y = df["Marks"]

# Step 3: Scatter Plot
plt.scatter(x, y, color="blue", marker="o")
plt.title("Scatter Plot of Marks vs Age")
plt.xlabel("Age")
plt.ylabel("Marks")
plt.grid(True)
plt.show()

# Step 4: Line Plot
plt.plot(x, y, color="green", marker="s", linestyle="--")
plt.title("Line Plot of Marks vs Age")
plt.xlabel("Age")
plt.ylabel("Marks")
plt.grid(True)
plt.show()
```

Sample Output (Graphical)

Result

Thus, the Python program successfully selected two columns from a DataFrame and plotted **scatter** and **line plots** to visualize the relationship between them.